

PRICING, RISK & TRUST ANALYTICS

A Power BI Analytics Project on E-Commerce Pricing Integrity

Internship Data Analytics Submission

Built on the same dataset and project established during training

Live Site: <https://boyfromthefaridabad.github.io/snapdeal-project/>

GitHub Repository: <https://github.com/boyfromthefaridabad/snapdeal-project>

Dataset (Google Drive): <https://docs.google.com/spreadsheets/d/1I5ursuFEcW3wcc5ma1ZW9hoSbirx1-Rd/edit>

1. Project Overview

This project analyzes a 220-product e-commerce catalogue across six categories (Electronics, Fashion, Home & Kitchen, Beauty & Personal Care, Sports & Fitness, and Grocery & Gourmet), paired with a 120-day daily price/discount/promotion scrape log (26,400 records), to detect where pricing, ratings, inventory risk, and customer trust are misaligned. All six tasks below extend the same dataset and Power BI file used during the training phase of this internship — no new dataset or project was introduced.

1.1 Data Model

Two tables form a star schema:

- Products — one row per product: Category, Brand, MRP, Rating, ReviewCount, ReturnRate, StockQuantity, StockValue, Sales_Last30Days, AvgDiscount_Last30Days.
- DailyScrapeLog — one row per Product per ScrapingDate: DiscountPercentage, SellingPrice, IsPromotional flag, UnitsSold.

Relationship: Products[ProductID] (1) → DailyScrapeLog[ProductID] (*). DailyScrapeLog[ScrapingDate] is used as the time dimension for all time-intelligence measures.

1.2 Tools

- Power BI Desktop — data modeling, DAX measures, dashboard visuals
- Python (pandas/numpy) — synthetic dataset generation matching realistic e-commerce distributions
- GitHub + GitHub Pages — version control and live hosting
- Google Drive — dataset file hosting for submission

Task 1 — Pricing vs Satisfaction Exception Detection

Objective

Identify, using measures only (no calculated columns), products that are simultaneously priced above their category average, rated below their category average, in the top 20% of Risk Score, and selling below their category's median — i.e., a compounding four-way failure, not a single weak metric.

DAX Measures

```
Avg Price in Category =  
CALCULATE ( AVERAGE ( Products[MRP] ), ALLEXCEPT ( Products,  
Products[Category] ) )  
  
Avg Rating in Category =  
CALCULATE ( AVERAGE ( Products[Rating] ), ALLEXCEPT ( Products, Products[Category]  
) )  
  
Median Sales in Category =  
CALCULATE ( MEDIANX ( ALLEXCEPT ( Products, Products[Category] ),  
Products[Sales_Last30Days] ) )  
  
Risk Score =  
Products[AvgDiscount_Last30Days] * ( 1 - ( Products[Rating] / 5 ) ) *  
Products[StockValue]  
  
Risk Score Percentile Rank =  
RANKX ( ALL ( Products ), [Risk Score], , DESC ) / COUNTROWS ( ALL ( Products ) )  
  
Is Exception Product =  
VAR PriceHigh = Products[MRP] > [Avg Price in Category]  
VAR RatingLow = Products[Rating] < [Avg Rating in Category]  
VAR RiskTop20 = [Risk Score Percentile Rank] <= 0.20  
VAR SalesBelowMedian = Products[Sales_Last30Days] < [Median Sales in Category]  
RETURN IF ( PriceHigh && RatingLow && RiskTop20 && SalesBelowMedian, 1, 0 )
```

Visual

Table visual: ProductName, Category, MRP vs Avg Price in Category, Rating vs Avg Rating in Category, Risk Score, Sales_Last30Days vs Median Sales in Category — filtered to [Is Exception Product] = 1.

Business Interpretation

These products fail on every dimension at once: customers pay a premium for something rated worse than category peers, the capital tied up in the stock is disproportionately risky, and sales aren't compensating for any of it. This is a compounding failure pattern, not a single bad metric, which is why it warrants a dedicated exception table rather than a KPI.

Recommended action depends on root cause: re-price when the rating gap is small and the core issue is competitive positioning; invest in quality improvement when rating gaps are large and return rates are elevated, since further discounting would only accelerate loss-making sales; and remove/delist when both price and rating gaps are severe and stock value is high, since the product is actively destroying margin and brand trust at scale.

Task 2 — Promotion Effectiveness Trend Analysis

Objective

Using ScrapingDate, calculate a baseline discount from the last 30 non-promotional days and compare it against daily discount levels, isolating the incremental lift attributable to promotional periods.

DAX Measures

```
Baseline Discount (30d non-promo) =  
VAR NonPromoDates =  
    CALCULATETABLE ( VALUES ( DailyScrapeLog[ScrapingDate] ),  
        DailyScrapeLog[IsPromotional] = 0 )  
VAR Last30NonPromo = TOPN ( 30, NonPromoDates, DailyScrapeLog[ScrapingDate],  
    DESC )  
RETURN  
CALCULATE (  
    AVERAGE ( DailyScrapeLog[DiscountPercentage] ),  
    KEEPFILTERS ( Last30NonPromo ),  
    DailyScrapeLog[IsPromotional] = 0  
)  
  
Daily Discount = AVERAGE ( DailyScrapeLog[DiscountPercentage] )  
  
Promotional Lift =  
VAR CurrentDiscount = [Daily Discount]  
VAR Baseline = [Baseline Discount (30d non-promo)]  
RETURN IF ( ISBLANK ( CurrentDiscount ) || ISBLANK ( Baseline ), BLANK(),  
    CurrentDiscount - Baseline )
```

Visual

Line chart — X axis: ScrapingDate; series: Daily Discount, Baseline Discount (30d non-promo), Promotional Lift — with a product/category slicer, since baseline discount is product-specific.

Business Interpretation

The baseline answers 'what does this product normally cost after typical discounting?', and the lift isolates the incremental discount attributable specifically to the promotion.

When promotional lift is large and coincides with a genuine spike in UnitsSold, the promotion is generating real incremental demand. When the 'baseline' price was already quietly rising in the days just before a promotion, the large discount is partly an illusion — customers aren't actually paying less than they would have a few weeks earlier. Distinguishing these two patterns matters because promotions that only simulate savings erode customer trust once noticed, and inflate reported promotional ROI without a real margin or demand benefit.

Task 3 — Discount vs Rating Causation Analysis

Objective

Test whether discounting causes better ratings, or whether the apparent relationship is confounded by review/sales volume, using a regression scatter plus review-volume-controlled comparisons.

DAX Measures

```
Avg Discount = AVERAGE ( DailyScrapeLog[DiscountPercentage] )
Avg Rating = AVERAGE ( Products[Rating] )
Review Count = SUM ( Products[ReviewCount] )

-- Review Volume Band (grouping): Low (<50), Medium (50-500), High (500+)
Avg Rating by Review Band =
CALCULATE ( AVERAGE ( Products[Rating] ) ) -- sliced by Review Volume Band on
the visual axis

Correlation Discount-Rating (approx) =
VAR N = COUNTROWS ( Products )
VAR SumX = SUMX ( Products, Products[AvgDiscount_Last30Days] )
VAR SumY = SUMX ( Products, Products[Rating] )
VAR SumXY = SUMX ( Products, Products[AvgDiscount_Last30Days] * Products[Rating] )
VAR SumX2 = SUMX ( Products, Products[AvgDiscount_Last30Days] ^ 2 )
VAR SumY2 = SUMX ( Products, Products[Rating] ^ 2 )
VAR Numerator = N * SumXY - SumX * SumY
VAR Denominator = SQRT ( ( N * SumX2 - SumX ^ 2 ) * ( N * SumY2 - SumY ^ 2 ) )
RETURN DIVIDE ( Numerator, Denominator )
```

Visual

(1) Scatter plot: X = Avg Discount, Y = Rating, with Power BI's built-in trend line enabled. (2) Supporting scatter: X = Avg Discount, Y = Review Count. (3) Bar chart: Avg Rating by Review Volume Band (Low/Medium/High).

Business Interpretation

If the raw scatter shows a mild positive slope between discount and rating, but that slope flattens once compared within the same review-volume band, the apparent relationship is a confound: heavier discounting drives more purchases, which drives more reviews, and high-volume products regress toward an average rating simply through statistical averaging — not because the product is genuinely better.

If discount correlates strongly with review count but only weakly with rating itself, that is direct evidence that ratings are improving mainly through volume dynamics rather than true satisfaction gains. The practical conclusion: 'discounted products have higher ratings' should not be read as proof that discounting improves quality perception — it is more likely a sampling effect that a manager should not use to justify discount-driven quality claims.

Task 4 — Financially Risky Inventory KPI

Objective

Define a Risk Score combining discount depth, rating shortfall, and stock value, then surface what share of total inventory value is financially at risk via a single KPI card with traffic-light conditional formatting.

DAX Measures

```
Risk Score =  
Products[AvgDiscount_Last30Days] * ( 1 - ( Products[Rating] / 5 ) ) *  
Products[StockValue]  
  
Total Stock Value = SUM ( Products[StockValue] )  
  
Risk Threshold (P75) = PERCENTILEX.INC ( ALL ( Products ), [Risk Score], 0.75 )  
  
At-Risk Stock Value (P75+) =  
SUMX ( FILTER ( Products, [Risk Score] >= [Risk Threshold (P75)] ),  
Products[StockValue] )  
  
% Inventory Value At Risk =  
DIVIDE ( [At-Risk Stock Value (P75+)], [Total Stock Value] )
```

Visual

KPI Card visual bound to % Inventory Value At Risk, with conditional background formatting: green below 10%, yellow 10-25%, red above 25% (implemented via a supporting color-state measure).

Business Interpretation

The Risk Score deliberately multiplies three failure modes together — discount depth, rating shortfall, and capital tied up as stock — so a product can be heavily discounted and still be fine if it's well rated and holds little inventory. The KPI only flags capital that is both being given away in discounts and attached to a poorly-reviewed item.

A rising red KPI signals the company is simultaneously mispricing (heavy discounting was required to move stock) and carrying quality problems at scale — both of which directly erode gross margin and long-run brand equity if left unaddressed.

Task 5 — Trust-Weighted Rating Index

Objective

Develop a Trust Score that discounts star ratings by how much evidence backs them (review volume) and how often customers returned the product, then compare a trust-weighted rating against simple and review-weighted averages.

DAX Measures

```
Trust Score =  
Products[Rating] * LOG ( 1 + Products[ReviewCount] ) * ( 1 -  
Products[ReturnRate] )  
  
Trust-Weighted Avg Rating =  
DIVIDE (   
    SUMX ( Products, Products[Rating] * [Trust Score] ),  
    SUMX ( Products, [Trust Score] )  
)  
  
Simple Avg Rating = AVERAGE ( Products[Rating] )  
  
Review-Weighted Avg Rating =  
DIVIDE (   
    SUMX ( Products, Products[Rating] * Products[ReviewCount] ),  
    SUM ( Products[ReviewCount] )  
)
```

Visual

Clustered column chart or three KPI cards side by side comparing Simple Avg Rating, Review-Weighted Avg Rating, and Trust-Weighted Avg Rating, optionally sliced by Category.

Business Interpretation

A product can post a high simple average rating from a handful of reviews with no returns data behind it — that number is fragile and easy to game. $\text{LOG}(1 + \text{ReviewCount})$ rewards ratings backed by real sample size but with diminishing returns, so products with very high review counts don't dominate the index purely on volume. Multiplying by $(1 - \text{ReturnRate})$ penalizes products customers rated well before actually using them, then sent back.

The result: a product with a perfect 5.0 rating from only 4 reviews and a 30% return rate is less trustworthy than a 4.3-star product with 3,000 reviews and a 4% return rate — and the Trust-Weighted Index surfaces that difference, while a naive star-rating leaderboard would not.

Task 6 — Context-Aware Price Banding

Objective

Classify products as Low, Medium, or High price bands using the 30th and 70th percentiles of price rather than fixed thresholds, so the boundaries recalculate automatically as category, brand, or time slicers are applied.

DAX Measures

```
Price Band (calculated column, static reference) =
VAR P30 = PERCENTILEX.INC ( ALLSELECTED ( Products ), Products[MRP], 0.30 )
VAR P70 = PERCENTILEX.INC ( ALLSELECTED ( Products ), Products[MRP], 0.70 )
RETURN SWITCH ( TRUE(), Products[MRP] <= P30, "Low", Products[MRP] <= P70,
"Medium", "High" )

-- Live, filter-context-aware measures (recompute per slicer selection):
Dynamic P30 Threshold = PERCENTILEX.INC ( ALLSELECTED ( Products ), Products[MRP],
0.30 )
Dynamic P70 Threshold = PERCENTILEX.INC ( ALLSELECTED ( Products ), Products[MRP],
0.70 )

Product Count by Dynamic Band =
VAR P30 = [Dynamic P30 Threshold]
VAR P70 = [Dynamic P70 Threshold]
RETURN SWITCH (
    TRUE(),
    SELECTEDVALUE ( 'Price Band Table'[Band] ) = "Low",
        COUNTROWS ( FILTER ( ALLSELECTED ( Products ), Products[MRP] <= P30 ) ),
    SELECTEDVALUE ( 'Price Band Table'[Band] ) = "Medium",
        COUNTROWS ( FILTER ( ALLSELECTED ( Products ), Products[MRP] > P30 &&
Products[MRP] <= P70 ) ),
    COUNTROWS ( FILTER ( ALLSELECTED ( Products ), Products[MRP] > P70 ) )
)
```

Visual

Stacked column or donut chart of product count (or stock value) by Dynamic Price Band, driven by a small disconnected 'Price Band Table' (Low/Medium/High), with Category and Brand slicers on the page to demonstrate live recalculation.

Business Interpretation

A fixed price band makes no sense across categories spanning very different price scales — a threshold that's 'expensive' for Grocery is trivially cheap for Electronics, so a static band mislabels almost everything outside the category it was tuned for. Percentile-based bands ask 'cheap or expensive relative to what's currently in view,' so filtering to a single category recomputes the 30th/70th percentile within that category, correctly reclassifying products that looked 'High' against the whole catalog but are mid-range within their own category.

This filter-context sensitivity is what makes DAX percentile measures more honest than hardcoded thresholds for real merchandising decisions — a manager filtering into one category needs bands calibrated to that category, not the whole catalog.

Conclusion

Across all six analyses, the same underlying principle recurs: single metrics lie by omission. A high rating can hide a high return rate; a large discount can hide an inflated baseline price; a low price band can be wrong the moment you filter into a different category. The measures built in this project are designed to catch those blind spots by combining signals — price, rating, risk, trust, and time — rather than reading any one of them in isolation, which is exactly the discipline an e-commerce manager needs to protect both profitability and brand trust.

Submission Checklist

- Live hosted website URL — GitHub Pages site (see docs/index.html in repository)
- GitHub repository — code, DAX measures, README, and this report
- Google Drive folder — Ecommerce_Dataset.xlsx and CSV exports
- Power BI (.pbix) file — built from this dataset with all six dashboards